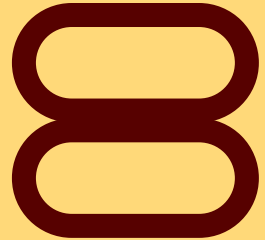
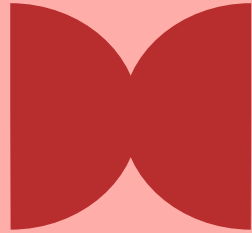


Automata Theory M26



Tutorial 1

Topics:

- Minimization of DFA
- NFA to DFA conversion

Recap – DFA and Regular Languages

What *is* all this?

DEFINITION 1.5

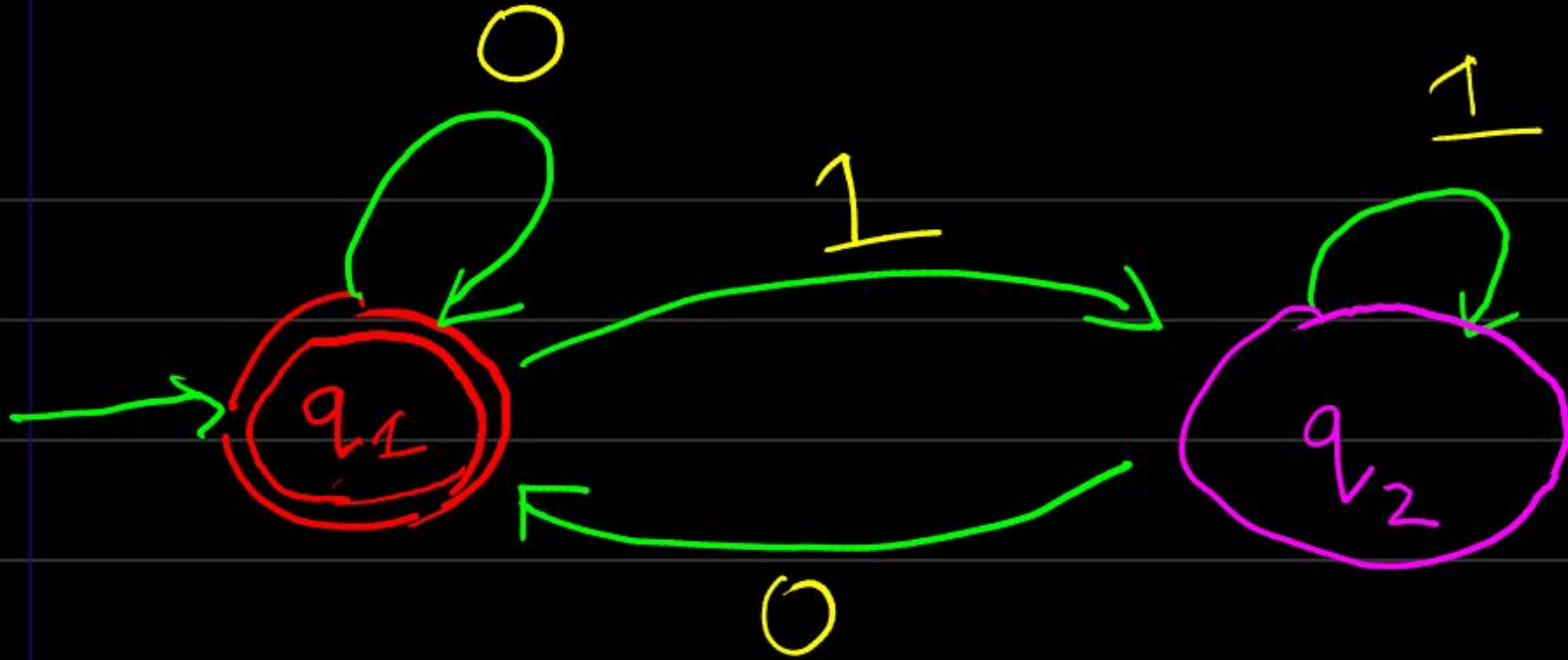
A *finite automaton* is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set called the *states*,
2. Σ is a finite set called the *alphabet*,
3. $\delta: Q \times \Sigma \longrightarrow Q$ is the *transition function*,¹
4. $q_0 \in Q$ is the *start state*, and
5. $F \subseteq Q$ is the *set of accept states*.²

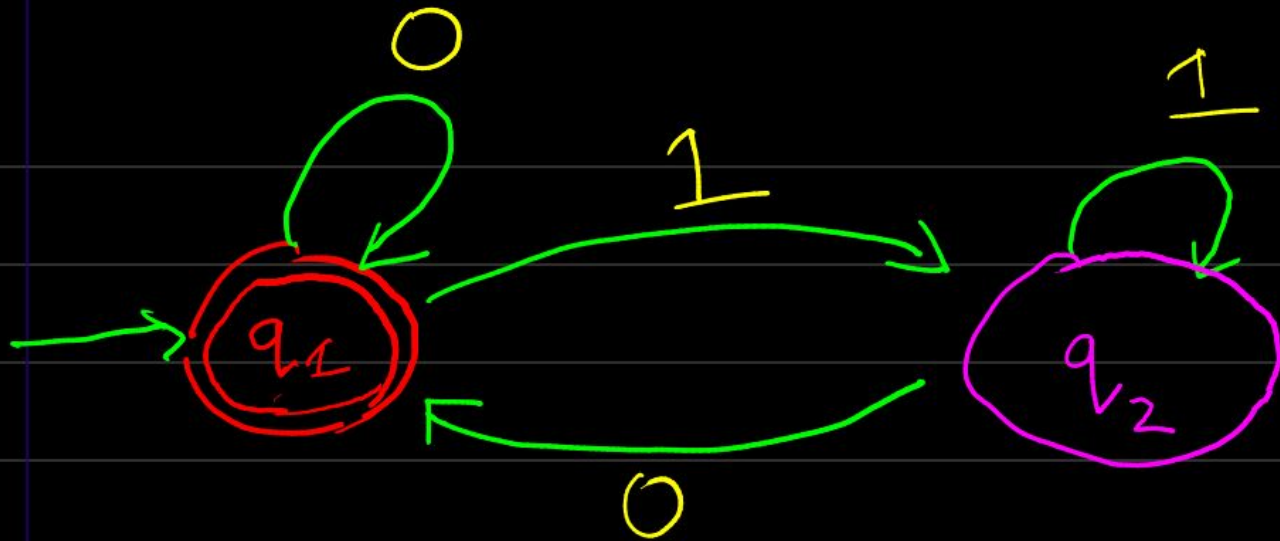
- When you load into a game.
- You begin on the Start State. Arrow keys become your E (alphabet).
- There is deterministic behaviour uniquely defined for each combination of – arrow key and tile that you currently are on.



What Language does this F.A describe? (We'll come back to this)

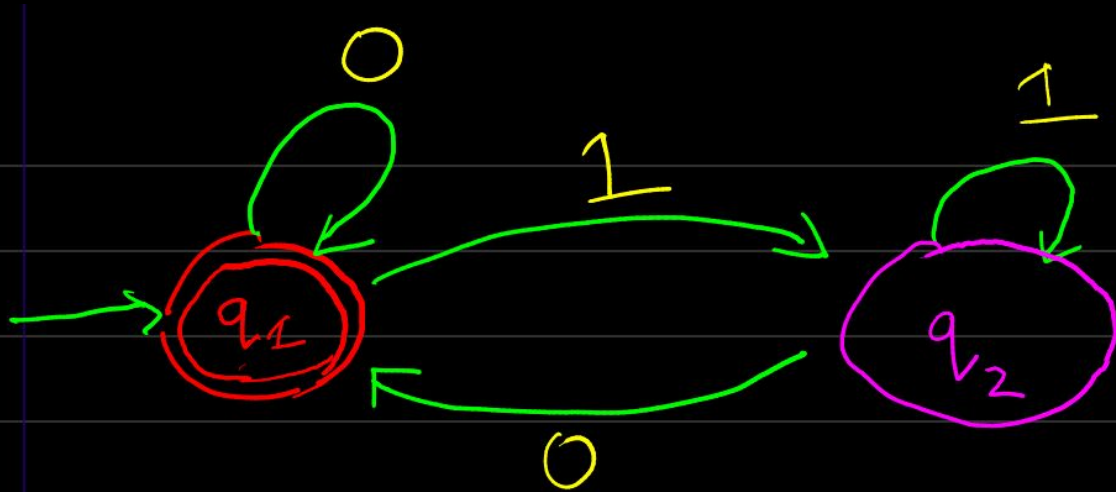


The 5 Tuple - What it is.



$Q = \{ q_1, q_2 \}$ Your 'STATES'

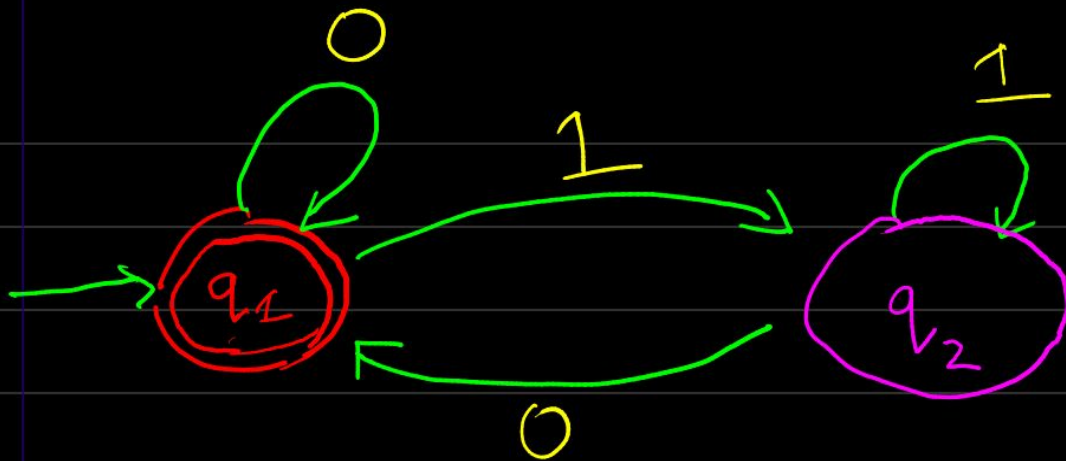
The 5 Tuple - What it is.



Σ , the alphabet = $\{0, 1\}$

( in a game controller)

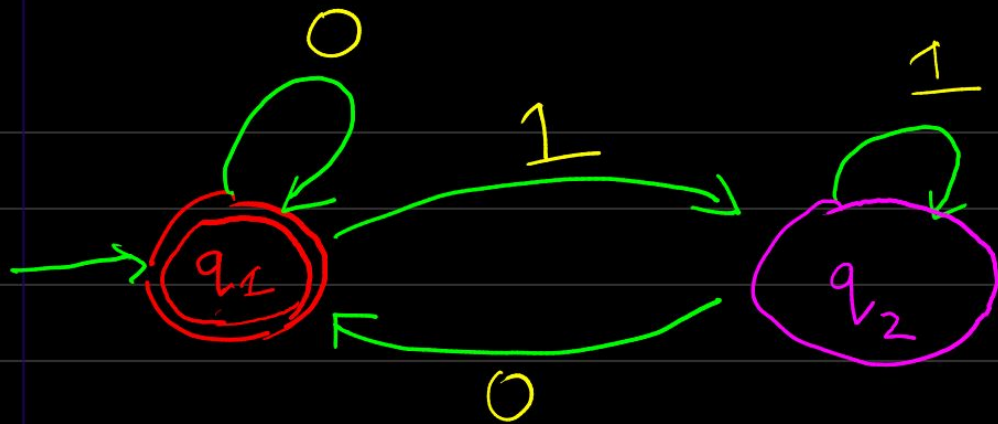
The 5 Tuple - What it is.



$\delta \rightarrow$ transition function

if I'm standing on q_x and get an alphabet,
WHERE to go??

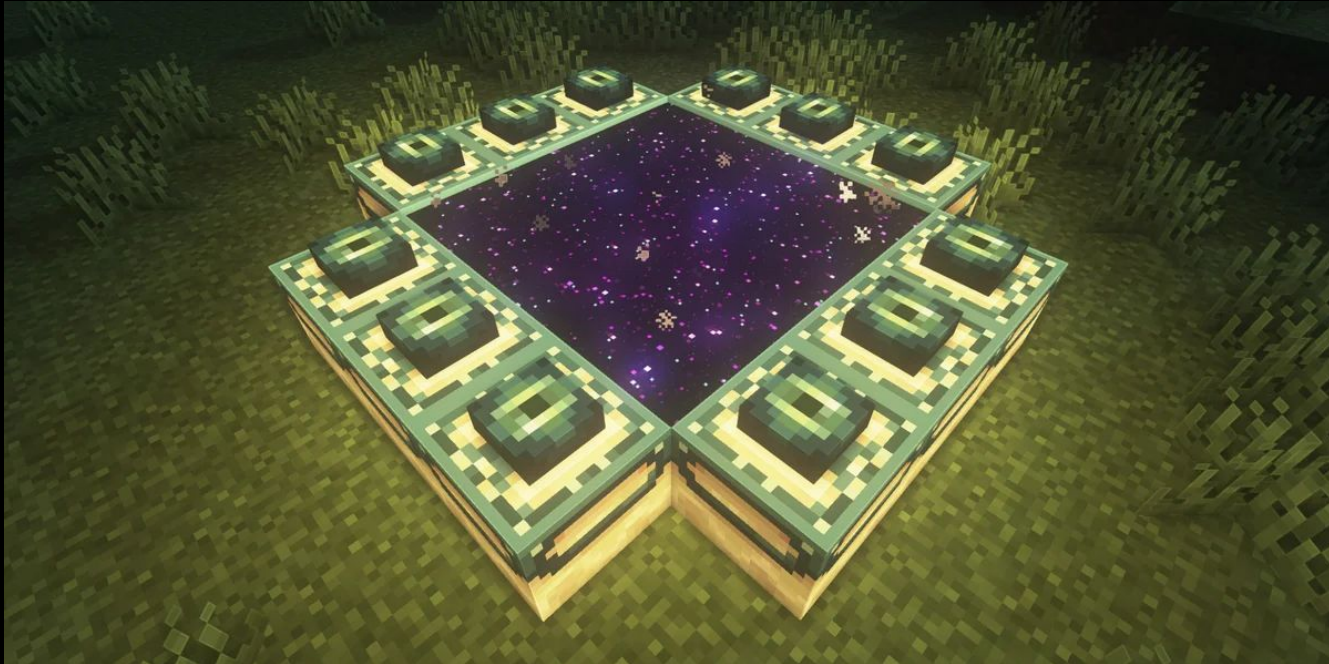
The 5 Tuple - What it is.



→ $q_1 \in Q$ start state

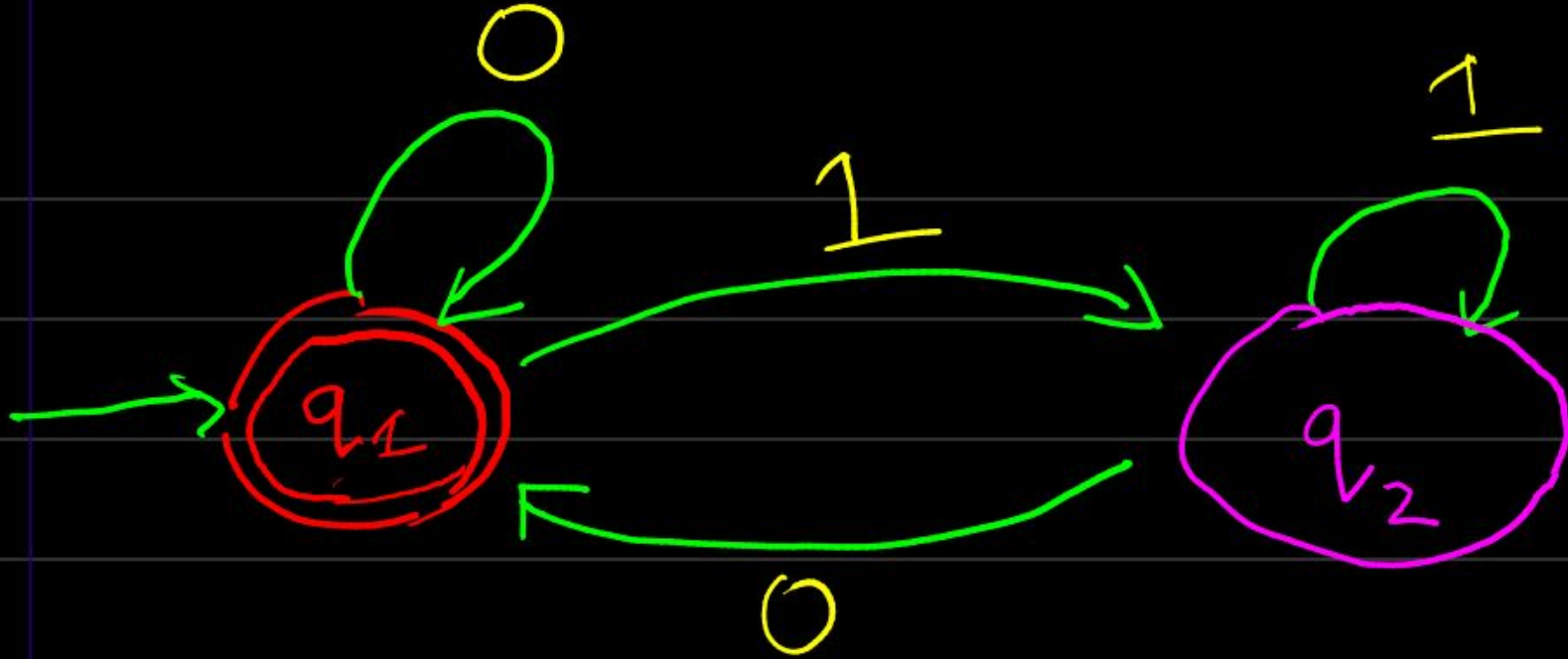
→ $F \subseteq Q$ accepting states (PLURAL!)
 ↳ subset of Q can be 1, 2, 3... states

- If you land on an accepting state after a string of alphabets, the F.A 'accepts' the entire input string.
- Your Automaton has successfully checked if the input string belongs to the language L that it describes.



Speaking of Languages....

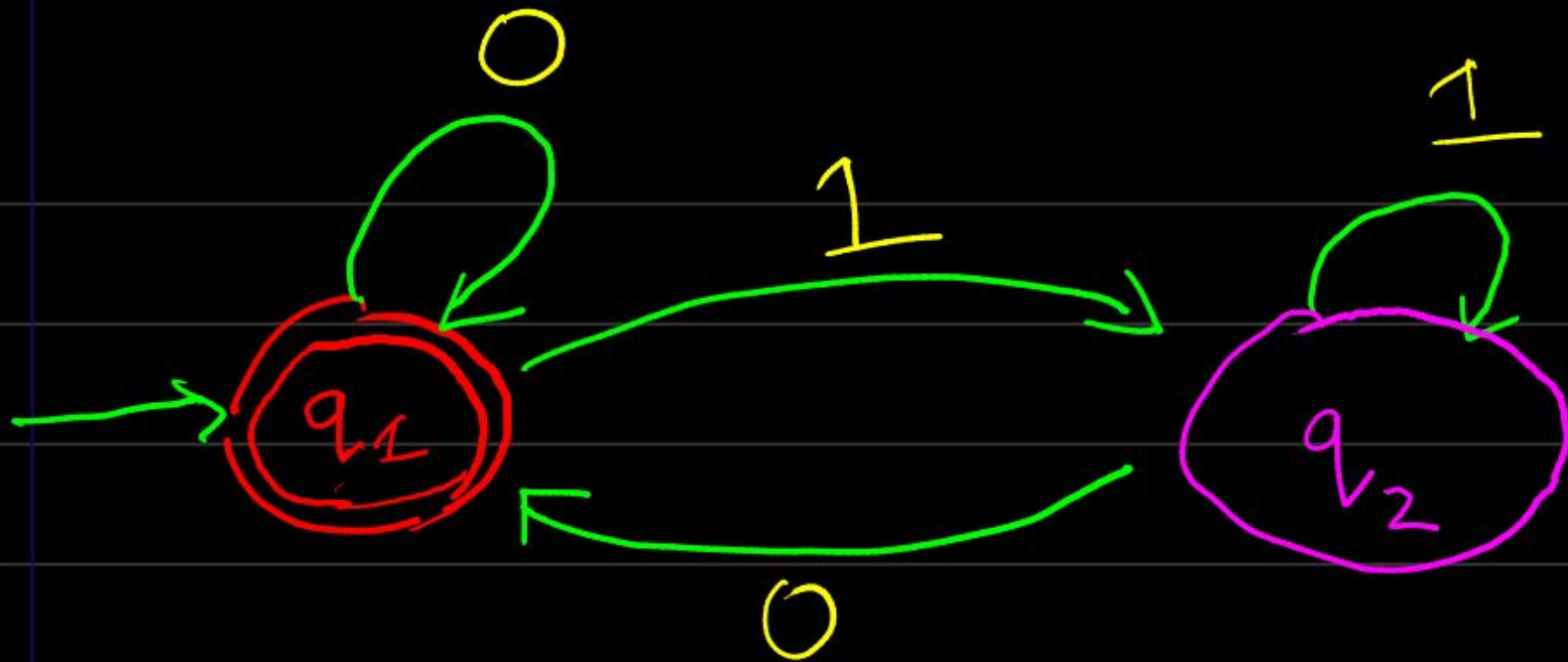
What Language does this F.A describe?



$L(M) = \{w \mid w \text{ ends in } 0\}$?

you missed something

What Language does this F.A describe?



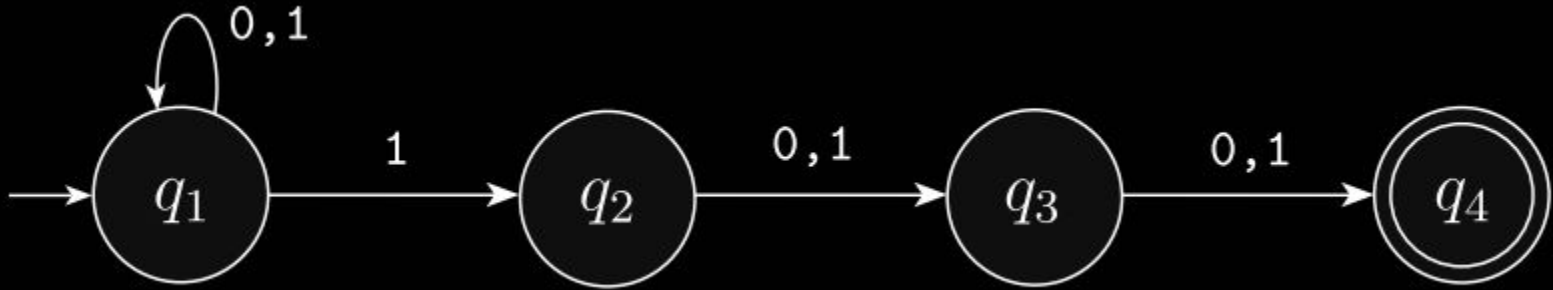
$$L(M) = \{w \mid w \text{ is the empty string } \epsilon \text{ or ends in a } 0\}.$$

DEFINITION 1.16

A language is called a *regular language* if some finite automaton recognizes it.

i.e DFAs and NFAs

NFAs – Nondeterminism



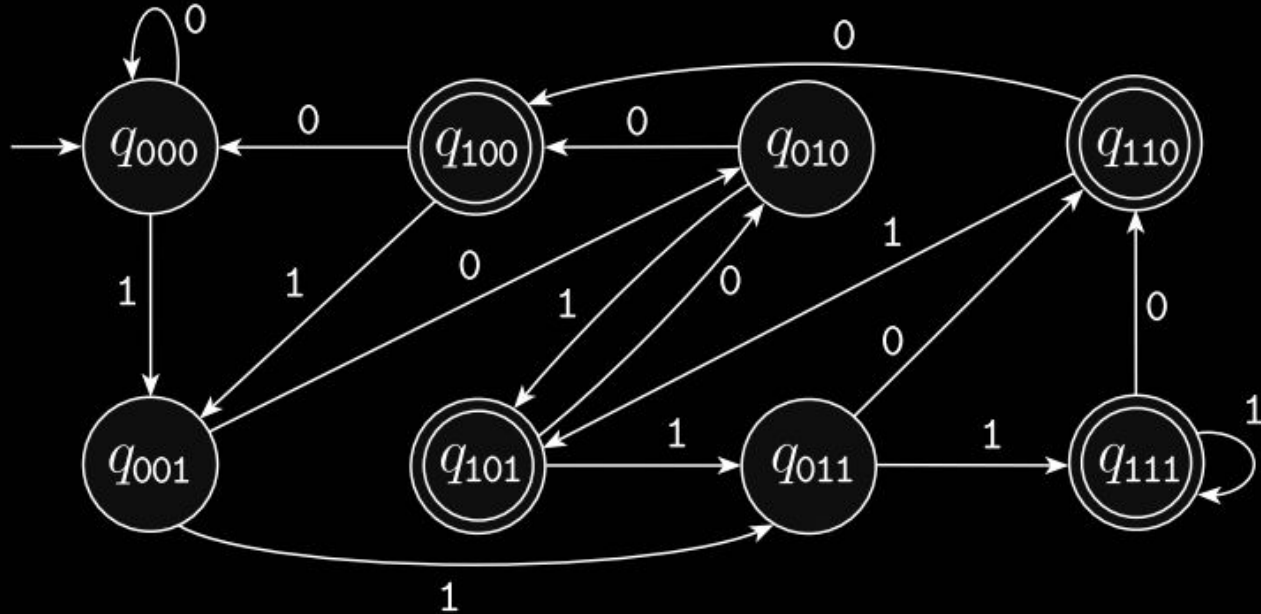
The key distinction from a DFA is -

For an input alphabet, you can arbitrarily 'pick' the next state.

(if even one of those paths leads to accept state, the string w belongs to L)

NFAs – Nondeterminism

The equivalent DFA



What changed?

DEFINITION 1.37

A *nondeterministic finite automaton* is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set of states,
2. Σ is a finite alphabet,
3. $\delta: Q \times \Sigma_{\varepsilon} \longrightarrow \mathcal{P}(Q)$ is the transition function,
4. $q_0 \in Q$ is the start state, and
5. $F \subseteq Q$ is the set of accept states.

The Transition Function

	0	1	ϵ
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset,$

In an NFA, from each state you can reach 'A SET' of other states on a particular input.

In a DFA, from each state you can reach only ONE other unique state for a specific input.

THEOREM 1.39

Every nondeterministic finite automaton has an equivalent deterministic finite automaton.

Minimization of a DFA – Distinguishability

Let $A \subseteq \Sigma^*$ and $x, y \in \Sigma^*$.

$x \equiv_A y$ iff for every $z \in \Sigma^*$, $xz \in A \iff yz \in A$

Otherwise x and y are **distinguishable**, and any z with $xz \in A \not\Rightarrow yz \in A$ distinguishes them.

Meaning. x and y are interchangeable — no future input can tell them apart.

$\equiv_A$ is an equivalence relation, so it splits Σ^* into classes.

Equivalence Relation

A relation R on a set S is an **equivalence relation** if it is:

1. **Reflexive** — xRx for every $x \in S$
2. **Symmetric** — $xRy \implies yRx$
3. **Transitive** — xRy and $yRz \implies xRz$

Index(L)
is simply the number of equivalence classes in L

Minimization of a DFA – Distinguishability

States p and q are **distinguishable** if there exists a string z such that exactly one of $\hat{\delta}(p, z), \hat{\delta}(q, z)$ is accepting.
Otherwise they are **equivalent**, written $p \sim q$.

The procedure of minimization will always result in a DFA

Myhill Nerode Theorem

For $A \subseteq \Sigma^*$ and $x, y \in \Sigma^*$:

$$x \equiv_A y \quad \text{iff} \quad \forall z \in \Sigma^* : xz \in A \iff yz \in A$$

$\equiv_A$ is an equivalence relation on Σ^* . Its number of equivalence classes is $\text{index}(A)$.

Theorem

$$A \text{ is regular} \iff \text{index}(A) < \infty$$

and in that case the minimal DFA for A has exactly $\text{index}(A)$ states.

Myhill Nerode Theorem

$$A \text{ is regular} \iff \text{index}(A) < \infty$$

- Every DFA for a language A has at least $\text{index}(A)$ number of states.
- A state of the minimal DFA is an equivalence class.
- Infinite index $\Rightarrow$ not regular.
- It is an if-and-only-if. Unlike pumping lemma it *proves* a language is regular

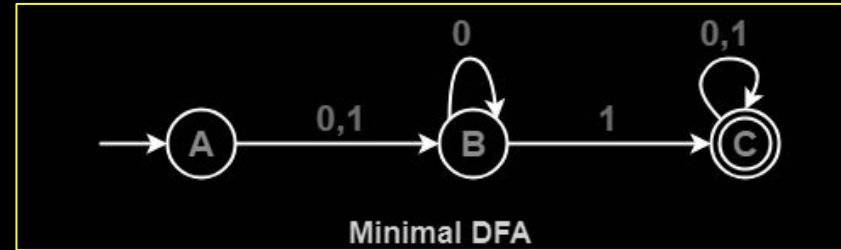
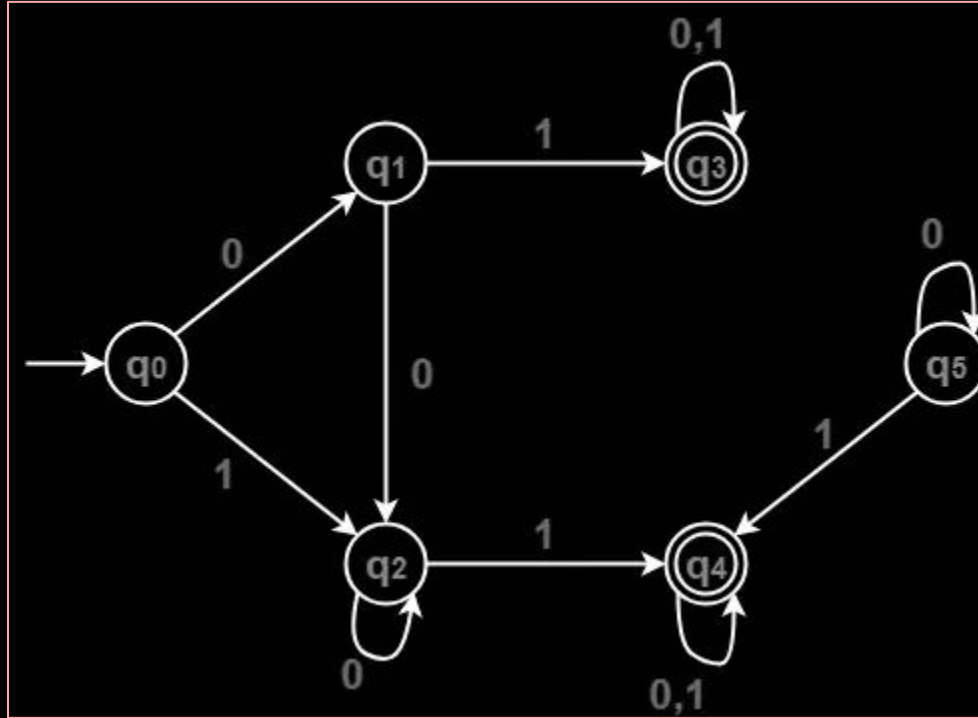
- 1.51** Let x and y be strings and let L be any language. We say that x and y are *distinguishable by L* if some string z exists whereby exactly one of the strings xz and yz is a member of L ; otherwise, for every string z , we have $xz \in L$ whenever $yz \in L$ and we say that x and y are *indistinguishable by L* . If x and y are indistinguishable by L , we write $x \equiv_L y$. Show that $\equiv_L$ is an equivalence relation.

Try going through these from Sipser

PROBLEMS 91

- ^{A*}**1.52 Myhill–Nerode theorem.** Refer to Problem 1.51. Let L be a language and let X be a set of strings. Say that X is *pairwise distinguishable by L* if every two distinct strings in X are distinguishable by L . Define the *index of L* to be the maximum number of elements in any set that is pairwise distinguishable by L . The index of L may be finite or infinite.

Minimization of a DFA



These two are the same!

Minimization of a DFA

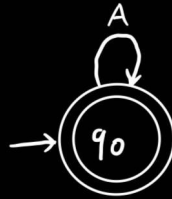
Case	On 0	On 1	Verdict
1	same group	same group	no split
2	same group	different groups	SPLIT
3	different groups	same group	SPLIT
4	different groups	different groups	SPLIT

- Here the 'groups' you should be checking... are simply the previous equivalence classes.
- Start with non-final and final states in two separate equivalence classes
- Split the equivalence classes till no more splits happen
- You'll be left with the states for your minimized DFA

practice practice practice

Thank you!

Call me



because the only Language I speak is
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA/